Syntax Choices for Generalized Pack Declaration and Usage

Document #:	P2671R0
Date:	2022-10-15
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Introduction

[P1858R2] proposed a lot of new facilities for working with packs and tuples. This paper is intended to be a companion paper to that one (it's kind of like an R3), but is focused solely on syntax decisions.

That paper roughly provided three kinds of facilities:

1. allowing declaring packs in more places
2. allowing more functionality for packs (indexing and slicing)

3. allowing more functionality for tuples (indexing and unpacking)

These facilities had varying levels of support, and I think it's worth exploring the various syntax options in more detail.

§ 2 Packs in More Places

[P1858R2] proposed allowing pack declarations in more contexts: more importantly allowing non-static data member packs:

```
template <typename... Ts>
struct tuple {
    Ts... elems; // proposed
};

template <typename... Ts>
void foo(Ts... ts) {
  using... Pointers = T*; // proposed
  Pointers... ps = &ts;   // proposed
}
```

Here, the syntax is basically already chosen for us. The syntax for member pack declaration has to be that, and its semantics follow pretty straightforwardly from all the other pack rules.

The local pack declarations are more interesting in that we have the question of what goes on the right-hand-side:

1. `auto... xs = ys;` (right-hand side is an unexpanded pack)
2. `auto... xs = ys...;` (right-hand side is an expanded pack)
3. `auto... xs = {ys...};` (right-hand side is an expanded pack within braces)

Here, we have precedent from [P0780R2] for option 1, since in lambdas we'd write `[...xs=ys]{}`. Option 2 here is a bit jarringly different, although option 3 is attractive for its ad hoc introduction of pack literals. The original paper briefly discussed this and it also has interesting interplay with what to do about expansion statements [P1306R1].[1]

I think we have to support option 1, and we should seriously consider option 3.

§ 2.1 Disambiguating Nested Packs

Once we allow declaring a member pack, we have to deal with the problem of disambiguation. Since if I pass an instance of the earlier `tuple` to a function template, and that template wants to expand the `elems` member, there needs to be some marker for that:

```
template <typename Tuple>
auto sum_tuple(Tuple tuple) -> int {
    // this can't work as-is
```

```
    return (tuple.elems + ...);
}
```

[P1858R2] proposed leading ellipsis for this (because once you start dealing with dots, everything is dots):

```
template <typename Tuple>
auto sum_tuple(Tuple tuple) -> int {
    return (tuple. ...elems + ...);  // ok
}
```

There aren't too many other options here - we need some marker to either put in front or behind `elems`. Using the word `pack` might be nice, as in `tuple.pack elems...` - that's probably a viable context-sensitive parse, since that is currently nonsense. Something longer like `tuple.packname elems...` seems a bit much.

2.2 Packs Outside of Templates

As [P2277R0] points out, once you allow a member pack declaration, you can get packs outside of templates:

```
template <typename... Ts>
struct simple_tuple {
  Ts... elems;
};

int sum(simple_tuple<int, int> xs) {
  return (xs.elems + ... + 0);
}
```

This proves concerning for implementations and led to the desire for some kind of prefix to indicate that somewhere a pack expansion follows. The question of course is… what prefix?

If we went back in time and made pack expansion a *prefix* `...` rather than a postfix `...`, this wouldn't have been a problem. For instance:

```
int call_f_with_squares(simple_tuple<int, int> xs) {
  return f(...(xs.elems * xs.elems));
}
```

This isn't sufficient for fold-expressions though, particularly since in `(E op ...)`, you don't get to the `...` again until the very end. `(... op E)` would be fine, but you can't just rewrite right folds into left folds - depending on the operator and the expression, those could evaluate very differently.

One option is to introduce a "pack expansion block" (credit to Davis Herring for this specific idea) - something like this:

```
int sum(simple_tuple<int, int> xs) {
  return ...{ (xs.elems + ... + 0) };
}

int call_f_with_squares(simple_tuple<int, int> xs) {
```

```
  return ... { f((xs.elems * xs.elems)...) };
}
```

Might be easier to see the distinctions if I line them up vertically:

```
// expansions
f(xs.elems...);           // status quo
f(...xs.elems);           // prefix expansion
...{ f(xs.elems...) }     // expansion block (with ... introducer)
expand { f(xs.elems...) } // expansion block (with keyword introducer)

// fold-expressions
(xs.elems + ...)             // status quo
...(xs.elems + ...)          // prefix expansion??
foldexpr (xs.elems + ...)    // fold-expression-specific introducer
... { (xs.elems + ... ) }    // expansion block (with ... introducer)
expand { (xs.elems + ... ) } // expansion block (with keyword introducer)
```

The nice thing about prefix expansion is that I think it's a boon not just to parsers but also to humans, and it also doesn't add additional characters. But there's no simple notion of prefix expansion for fold-expressions, the closest thing might be to just slap an extra `...` in front of the parentheses?

The expansion blocks (whether introduced with `...` or a hypothetical keyword that can't be as simple as `expand`) do add quite a bit of noise.[2] EWG wasn't a fan of using the `...` in particular, which isn't surprising, because the last thing that variadic code needs are *more* ellipses.

The advantage of the `expand` block though is that since the goal is just to, basically, turn on pack expansions, one such block can include arbitrarily expansions:

```
int f(simple_tuple<int, int> xs) {
  return expand { g(xs.elems...)  + (xs.elems + ...) };
}
```

Nevertheless, it seems like the right approach is to prefix-expand pack expansions and to come up with some introducer for fold-expressions. The former is *much* more common than the latter, so having to a pay a syntax penalty for the latter but not the former seems like a good tradeoff. I'm going to suggest `foldexpr`:

```
void func(simple_tuple<int, int> xs) {
  int a = call(xs.elems...);          // error
  int b = call(...xs.elems);          // ok
  int c = (... + xs.elems);           // ok
  int d = (xs.elems + ...);           // error
  int e = foldexpr (xs.elems + ...);  // ok
}
```

There's also a different approach. The difference between variadic templates (which have no idea for a prefix) and non-variadic contexts (which do) is precisely the fact that you don't know that somewhere a pack might appear. With a variadic template, you know this at the point of declaration. You see the `...`, but with a regular function, you don't. What if we just added a marker for to signal to the compiler that pack expansions are coming in this scope? That is, we don't touch any of the existing syntax for

expansion - it's just that we require that any expansion is preceded in scope by a visible declaration of a pack:

```
void func(simple_tuple<int, int> xs) {
  int a = call(xs.elems...);          // error
  int b = call(...xs.elems);          // error (pack prefix isn't a thing)
  int c = (... + xs.elems);           // error

  {
    using ...;                        // the packs are coming!
    int d = (xs.elems + ...);         // ok
    int e = call(xs.elems...);        // ok
    int f = call(...xs.elems);        // error (pack prefix still isn't a
        thing)
  }

  int g = call(xs.elems...);          // error (still)
  auto [...ys] = xs;                  // assuming P1061
  int h = call(ys...);                // ok (we've seen a pack declaration)
  int i = call(xs.elems...);          // ok (we've seen a pack declaration)
}
```

The `using ...` bit is a bit awkward and novel - but it allows us to retain existing rules for pack expansions and fold-expressions, and this rule would also limit potential changes once we get structured bindings in here too [P1061R2]. Worth considering.

3 More Functionality for Packs

There are four primitive operations for packs:

1. expanding
2. indexing
3. slicing (i.e. producing another pack out of the original one)
4. iterating

Today, we can only do one of them. [P1306R1] originally proposed handling iteration, but has had to walk away from that after ambiguity issues, and that paper has stalled too.

To explain why these are the primitives, we can go over what you can actually do with a pack.

You can consume it immediately, whether via `f(xs...)` or `(xs && ...)`, that's expansion. You can perform an operation on every element in a pack. You can't do this directly yet, but you can get a decent approximation by way of creating a lambda that defines the operation you want to do and then folding over a comma: `(f(xs), ...)`. I guess the comma is useful after all.

Or, you can pop one element off and then handle the rest separately - similar to how in Haskell you would do `x:xs` in an overload. Imagine wanting to print a pack, comma-delimited. You can't really do that with a fold-expression - since you want to do one operation for the first element and then a different

operation for the rest of the elements. That's indexing (for the head) and slicing (for the tail). A similar story holds for wanting to implement `visit` except putting the function last instead of first: the language gives us an easy way to split off the first element, but not so much for the last element. That's again indexing and slicing. Other situations might call for taking the first or last half of a pack to recurse.

3.1 Indexing

The issue with indexing is largely a choice of syntax. We can't use *`pack`*`[0]` because of potential ambiguity if this appears in a pack expansion: `call(x[0] + x...)` is valid syntax today, but this isn't asking for the first element of the pack `x`, it's indexing into every element of the pack `x`.

That leaves introducing some new syntax that is distinct. I think the options are here *`pack`*`.[0]` (as [N4235] suggested) or *`pack`*`...[0]` (as [P1858R2] suggested). Between *`pack`*`.[0]` and *`pack`*`...[0]`, I think the latter is better for two reasons:

1. It does seem like operations with packs should just use `...`, this one is no different
2. It looks like shorthand for `tuple(`*`pack`*`...)[0]`. The latter isn't valid yet (would require constexpr function parameters to be valid), but the similarity between the two expressions seems compelling to me anyway.

Note that this is a frequently desired utility, and there is a whole talk at CppNow specifically about how to efficiently index into a pack: The Nth Element: A Case Study.[3] Also note that Circle implements the *`pack`*`...[0]` syntax.

3.1.1 Type Indexing

Pack indexing shouldn't just work for a pack of values, it should also work for a pack of types. That would allow:

```cpp
template <typename... Ts>
auto first(Ts... ts) -> Ts...[0] {
  return ts...[0];
}
```

Packs of types follow the same principle as packs of values.

3.1.2 Indexing from the Back

Once we have indexing in general, a lot of the rules more or less follow. The index needs to be within the range of the size of the pack, and if not, that's an immediate-context error (`first()` above is ill-formed, since there is no first type, but it's ill-formed in a SFINAE-friendly way).

But there's still an interesting question here: how do you return the *last* element? Well, you could write this:

```cpp
template <typename... Ts>
auto last(Ts... ts) -> Ts...[sizeof...(Ts) - 1] {
```

```
  return ts...[sizeof...(ts) - 1];
}
```

This is… fine. This is fine. It works, it does the right thing. But it's a bit tedious.

Other languages provide a dedicated facility for counting backwards from the last element:

— Python uses `Ts...[-1]`

— D uses `Ts...[$-1]`

— C# uses `Ts...[^1]`

The problem with negative indexing is that while it's convenient most of the time, and is reasonably easy to understand and use, it does have surprising problems on the edges:

```
def last_n_items(xs, n):
    return xs[-n:]

last_n_items(range(10), 3) # [7, 8, 9]
last_n_items(range(10), 2) # [8, 9]
last_n_items(range(10), 1) # [9]
last_n_items(range(10), 0) # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Not to mention that if you do some math to determine an index, it'd be nice if overflowing past zero would be treated as an error rather than some wildly different meaning.

The D and C# approaches don't have this issue. But `$` seems like a waste of that token, which can be put to more interesting uses (although it has prior art in regex as well, also meaning the end). The C# approach clashes with reflection using `^` as the reflection operator (and, even if it didn't, seems like a waste of that token too). In both cases, this syntax can *only* appear inside of indexing (or slicing) expressions. But I think such a restriction would be fine. In both cases, had the Python code used these alternate syntaxes (returning either `xs[$-n:]` or `xs[^n:]`, as appropriate), then the first three calls would be equivalent while the last call, `last_n_items(range(10), 0)`, would return an empty list – which would be the correct answer.

A more C++ approach would be to introduce a new type that means from the end:

```
template <typename... Ts>
auto last(Ts... ts) -> Ts...[std::from_end{1}] {
  return ts...[std::from_end{1}];
}
```

This doesn't have either the issue with negative indexing or the issue with requiring a dedicated token.

Circle implements Python's approach.

3.1.3 Reflection

Reflection [P1240R2] does provide the facilities to do pack indexing, it would look something like this:

```
template <typename... Ts>
auto first(Ts... ts) {
```

```
  constexpr std::vector infos = {^ts...};
  return [: infos[0] :];
}
```

That is: can take the pack, produce a range of `meta::info` from it, and then splice the first one. This works,[4] and it's great that we will have such a facility. But it's a bit verbose for what I would consider to be a primitive. If we want to annotate the return type as well:

```
template <typename... Ts>
auto first(Ts... ts) -> [: std::vector{^Ts...}[0] :] {
  return [: std::vector{^ts...}[0] :];
}
```

This could be another function in the `std::meta` namespace, which would have to take its parameters by `std::meta::info` (so as to work for both packs of types and packs of values), but that doesn't actually end up being any shorter, and loses the `[]` part of the indexing, which I think is quite valuable:

```
-> [: std::vector{^Ts...}[0] :]           // directly
-> [: std::meta::select(0, {^Ts...}) :]   // with a function
```

I would personally always write the `vector` version directly.

§ 3.2 Slicing

Sometimes you want one element from a pack, sometimes you want multiple. That's slicing. There are basically two syntax I've seen languages use for slicing:

— `[from:to]`

— `[from..to]`

In both cases, `from` can be omitted (implicitly meaning `0`) and `to` can be omitted (implicitly meaning the end). That is, `[:]` or `[..]` would mean to take the entire pack.

The advantage of the former is that it uses fewer dots. And also that it also can be extended by another argument as `x[from:to:stride]` (e.g. `[::2]` would be taking every other element, starting from the first), although while slicing a pack does come up regularly, striding doesn't feel like it's that common.

The advantage of the latter is that it's probably more viable to in a for loop (e.g. `for (int i : 0..10)`, since having the extra colon would be fairly awkward) and it simply looks more like we're presenting a range.[5]

However, the issue with `0..10` is this is currently parses as a single *`pp-number`*, so there might need to be a bit more work to have this functional. I think it's a better syntax overall, so hopefully this doesn't prove too problematic.

Regardless of which syntax to choose, the arguments about indexing from the back still apply, as well as the syntax for how to slice a pack.

If we're going to index into a pack via `pack...[0]` then we should slice a pack via `pack...[1:]` (or `pack...[1..]`). And what we end up with, at this point is… still a pack. So it would need to be expanded.

For instance, one (not-great) way of writing `sum` might be:

```
template <class... Ts>
auto sum(Ts... ts) -> int {
  if (sizeof...(Ts) == 0) {
    return 0;
  } else {
    return ts...[0] + sum(ts...[1..]...);
  }
}
```

Or the comma-delimited print example I mentioned earlier:

```
template <typename... Ts>
void print(Ts... ts) {
  if constexpr (sizeof...(Ts) > 0) {
    // print the first element
    std::print("{}", ts...[0]);

    // then print every other element with preceding space
    auto f = [](auto e){ fmt::print(" {}", e); };
    (f(ts...[1..]), ...);
  }
}
```

That's a lot of dots in the last case, but it does work pretty well.

Slicing, while not as important as indexing, still is an operation that regularly comes up, so I think it would be important to support. Whichever syntax we choose for slicing a pack could also be used to slice other objects as well. If I have some `s` that is a `span<T>`, `s[2..4]` could conceivably be made to work (and evaluate basically as `s.subspan(2, 2)`, which we have today).

§ 3.3 Pack Object

The syntax I'm suggesting for pack indexing is `pack...[0]`. The reflection solution looks similar to that, just with a few more characters: `[: std::vector{^pack...}[0] :]`. But what if we could take what the reflection approach is doing and come up with something that gets there more directly?

Today, the only way to use a pack is to expand it immediately. But we know we need a way to do other things - iterate, index, slice. All things we can do with an object pretty easily. If we could annotate `pack` in such a way to make it clear that we're referring to the pack *itself*, as an entity, that could be interesting. Like `OBJECT(pack)` or `pack.into_object()`. But we need some kind of syntax to be *definitely* unambiguous and also *distinct* grammatically, which neither of those really allow for. We would need some sort of token.

One option that does work is `pack![0]`. The `!` cannot appear after an identifier today. This could be a postfix operator that can only follow the name of a pack, that gives you an object that behaves somewhat

similarly to `vector{^pack...}`, except instead of a container of `std::meta::info`, it behaves more like a container of expressions. There are a few other tokens that could be used here as well, but there's something kind of nice about `elems!` meaning "no, actually, this `elems` thing as a whole!" that I kind of like.

That is, it could give you an object that looks like this:

```
template <std::vector<std::meta::info> V>
struct PackObject {
  constexpr auto operator[](std::ptrdiff_t idx) const {
    return [: V[idx] :];
  }
};
```

With that, `pack![0]` would be the first expression in the pack.

This also provides an answer for expansion statements: an expansion statement only traverses an object, so if we want to expand a pack we have to first turn it into an object… via `pack!`:

```
template <typename... Ts>
void foo(Ts... ts) {
  // P1306: error, can't use a pack here
  template for (auto elem : ts) { ... }

  // P1306: ok, tuple is fine (though wasteful)
  template for (auto elem : std::tuple(ts...)) { ... }

  // P1306 with P1240: okay, but requires an extra step
  template for (constexpr auto i : std::vector{^ts...}) {
    auto elem = [:i:];
    ...
  }

  // This paper: use a pack object to iterate over the pack
  template for (auto elem : ts!) { ... }
}
```

Between `pack...[0]` and `pack![0]`, the former has the benefit of (some kind of) consistency with existing pack facilities, while the latter has the benefit of using fewer dots. There are a lot of dots when dealing with packs and it would be nice to have fewer of them. The downside though is that introducing more tokens pushes us further on the path to Perl, so there's no free lunch here either.

§ 3.3.1 Slicing a Pack Object

If we go with a pack object, then slicing might look a bit different. If `pack!` is an object, such that `pack![0]` is the first element, what would `pack![1..]` be? Whereas `pack...[1..]` would have to be an unexpanded pack, `pack![1..]` makes more sense to be… another pack object.

That pack object would need to be expanded. So in the same way that we need a token to treat a pack as an object, we'd need a token to treat an object as pack. Unpacking a pack object is the same kind of

operation as unpacking a `std::tuple` - an inline `std::apply`. Let's take `~` and reconsider the previous examples with slicing:

Indexing with `...[0]`	Indexing with a pack object

`template <class... Ts>`
`auto sum(Ts... ts) -> int {`
`  if (sizeof...(Ts) == 0) {`
`  return 0;`
`  } else {`
`  return ts...[0] + sum(ts...`
`  [1..]...);`
`  }`
`}`
	`template <class... Ts>`
	`auto sum(Ts... ts) -> int {`
	`  if (sizeof...(Ts) == 0) {`
	`  return 0;`
	`  } else {`
	`  return ts![0] + sum(ts![1..]~...);`
	`  }`
	`}`

`template <typename... Ts>`
`void print(Ts... ts) {`
`  if constexpr (sizeof...(Ts) > 0) {`
`  // print the first element`
`  std::print("{}", ts...[0]);`
 	
`  // then print every other element`
`  with preceding space`
`  auto f = [](auto e){ fmt::print("`
`  {}", e); };`
`  (f(ts...[1..]), ...);`
`  }`
`}`
	`template <typename... Ts>`
	`void print(Ts... ts) {`
	`  if constexpr (sizeof...(Ts) > 0) {`
	`  // print the first element`
	`  std::print("{}", ts![0]);`
	
	`  // then print every other element`
	`  with preceding space`
	`  auto f = [](auto e){ fmt::print("`
	`  {}", e); };`
	`  (f(ts![1..]~), ...);`
	`  }`
	`}`

Slicing also presents a good motivation for choosing *`pack![0]`* as the choice for indexing, because if slicing a pack involves ellipsis and then expanding that pack involves another ellipsis *and also* the slicing involves `..`, that's a tremendous amount of `.`s for a single expression.

§ 3.4 Summary

The two pack operations suggested here both have similar syntax. Either we expand the pack and index into it:

— *`pack`*`...[0]` gives me the first element

— *`pack`*`...[1..]` gives me every element starting with the second, and is still a pack

or we have a special syntax to identify that the pack should be treated as a distinct object and index into that:

— *`pack`*`![0]` gives me the first element

— *`pack`*`![1..]` gives me every element starting with the second, as a new pack object

— *`pack`*`!~` is the same pack as *`pack`*`!`

I think both syntax options work pretty well, and are consistent with how both packs and indexing work today.

Importantly, *`pack`* in the above should be an identifier that denotes a pack, not an arbitrary expression. Partially this avoids the question of: in `f(x)...[0]`, how many times is `f` invoked? But also because it's not strictly necessary anyway. If I want to invoke `f` on the first element, I could do `f(x...[0])`. If I wanted to invoke `f` on all the elements starting from the second, I don't need `f(x)...[1..]...`, I can just `f(x...[1..])...` It's the same amount of characters in both cases anyway, so there's really no reason to have to get complicated here.

4 More Functionality for Tuples

There are two things we can't easily do with tuples, but we should be able to:

1. unpacking
2. indexing

Indexing into a tuple is technically possible today, although the syntax at the moment is `std::get<0>(tuple)`, which is decidedly unlike any other indexing syntax in this or any other language. `boost::tuple` at least supports `tuple.get<0>()`, which puts the index last.

Unpacking is also technically possible today, by way of `std::apply`, but is extremely unergonomic to say the least.

4.1 Tuples <=> Packs

The syntax for indexing into a tuple and unpacking a tuple (that is, turning a tuple into an unexpanded pack referring to the elements of the tuple) should mirror the syntax for dealing with packs. That is:

Pack Syntax	Tuple Syntax

`pack...[0]`
`pack...[..]...`
`pack...[1..]...`
	`tuple.[0]`
	`tuple.[..]...;`
	`tuple.[1..]...;`

`pack![0]`
`pack![..]~...`
`pack![1..]~...`
	`tuple.[0];`
	`tuple~...;`
	`tuple.[1..]~...;`

A few things to note here. While the slicing syntax for the "whole slice" (whether `[..]` or `[:]`) is not especially useful if you're starting from a pack, it is, on the other hand, the most useful thing when dealing with a tuple. Since, when you're unpacking a tuple into a function, typically you'll want to unpack the entire tuple.

Given a syntax for unpacking a tuple, like `tuple.[..]...`, there doesn't need to be another syntax on top of that to index into a tuple. Since, once we have a pack, we can index into the pack with `tuple.[..]...[0]`. But that feels a bit excessive,[6] so having a shorthand for this case seems justified.

Now, if we have a dedicated postfix operator (`!`) to treat a pack as an object, then it might make sense to have a mirrored postfix operator (`~`) to treat an object as a pack. The same pros and cons here apply: this reduces the number of dots you have to write (which themselves hinder comprehension), but it increases the amount of punctuation required and brings us closer to Perl (which itself hinders comprehension). But if `tuple~` gives us a pack (which would be quite useful, as the common case of wanting to unpack a tuple is indeed to unpack the *entire* tuple, so having a short marker for this is quite nice), then how would you index into that resulting pack? Well, that would have to be `tuple~![0]` and `tuple~![1..]...`. But that's just awkward (we take our tuple, turn it into a pack, then turn it back into an object?), so I think it's worth still resorting to `tuple.[0]` in this case anyway. But `tuple~` is worth considering nevertheless.

Here's a concrete example of the difference between the `...[0]` and `![0]` indexing syntaxes for a tuple implementation that also does unpacking (imagine a `product` function taking a pack of integers and returning its product), using the terser `tuple~` form to unpack a tuple:

Indexing with `...[0]`

```
template <typename... Ts>
class tuple {
  Ts... elems;
public:
  tuple(Ts... ts) : elems(ts)... { }

  using ...tuple_element = Ts;

  template <size_t I>
  auto get() const& -> Ts...[I]
        const& {
    return elems...[I];
  }
};

int main() {
  tuple vals(1, 2, 3, 4);
  assert(vals.[0] + vals.[1] + vals.
        [2] + vals.[3] == 10);

  assert(product(vals.[..]...) ==
        24);
  assert(product(vals.[2..]...) ==
        6);
}
```

Indexing with a pack object

```
template <typename... Ts>
class tuple {
  Ts... elems;
public:
  tuple(Ts... ts) : elems(ts)... { }

  using ...tuple_element = Ts;

  template <size_t I>
  auto get() const& -> Ts![I] const& {
    return elems![I];
  }
};

int main() {
  tuple vals(1, 2, 3, 4);
  assert(vals.[0] + vals.[1] + vals.[2]
        + vals.[3] == 10);

  assert(product(vals~...) == 24);
  assert(product(vals.[2..]~...) == 6);

}
```

§ 4.2 Unified Operations

Treating a tuple as a pack is closely tied in with slicing a pack, so considering these operations together makes a lot of sense. I don't think they are meaningfully separable, as they have to inform each other.

And part of the value of having a dedicated syntax for pack/tuple indexing, pack slicing, and tuple unpacking, is precisely that these syntaxes can mirror each other… and the indexing operations for packs and tuples share a syntax with the indexing operations for other kinds. The slice syntax could be used in other contexts (perhaps `1..3` can be an expression of value `std::slice(1, 3, 1)`?[7]).

One of the push-backs against these facilities is that a hypothetical reflection facility could address these use-cases. And that's probably true, we could add different functions for each of these cases. But then we'd end up with differently named functions - losing the symmetry. I think that would be unfortunate.

§ 4.3 Nested Packs

[P1858R2] had a whole section on nested pack expansions. The model that paper suggested was that the `.[:]` operator, when applied to a tuple, would add a layer of packness. In the normal case, we go from "not a pack" to "a pack," but if we have a pack of tuples, we could then go to a pack of packs. The paper contained this example:

```
template <typename... Ts>
void foo(Ts... e) {
    bar(e.[:]... ...);
}

// what does this do?
foo(xstd::tuple{1}, xstd::tuple{2, 3});
```

And suggested that this calls `bar(1, 2, 3)`.

That's… cool. But it adds an unbelievable level of complexity that I don't think is really worth it, and surely necessitates a better syntax. Like having proper list comprehensions? So I don't think the above example should be valid, just for the forseeable future.

§ 4.4 Table of Disambiguation

[P1858R2] included a table of the various kinds of member-access expansions that could occur of the form `e.f`, where either `e` or `f` could be a pack, a tuple, or a simple object. Adjusted for the syntax presented here, and removing support for nested packs, that table now looks as follows.

If we use `...[0]` for pack indexing and `.[..]` for tuple unpacking:

	e is a Pack	**e is a Tuple**	**e is not expanded**
f is a Pack	not possible	not possible	`foo(e. ...f...);`
f is a Tuple	not possible	not possible	`foo(e.f.[..]...);`
f is not expanded	`foo(e.f...);`	`foo(e.[..].f...);`	`foo(e.f);`

If we use a pack object for pack indexing and `~` for tuple unpacking:

	e is a Pack	**e is a Tuple**	**e is not expanded**
f is a Pack	not possible	not possible	`foo(e.pack!(f)...);`

`f` is a Tuple	not possible	not possible	`foo(e.f~...);`
`f` is not expanded	`foo(e.f...);`	`foo(e~.f...);`	`foo(e.f);`

With either choice of syntax, the table is significantly reduced, since we no longer have to deal with the question of layering packs.

5 Proposal

I think the right set of functionality to have is:

— the ability to declare member packs, alias packs, and local variable packs

— the ability to index and slice into a pack, including from the back

— the ability to index into and unpack a tuple

I think there are two good choices of syntax here for indexing, slicing, and unpacking:

	Option 1	Option 2
Pack Indexing (first element)	`pack...[0]`	`pack![0]`
Pack Indexing (last element)	`pack...[std::from_end{1}]`	`pack![std::from_end{1}]`
Pack Slicing	`pack...[1..]...`	`pack![1..]~...`
Tuple Indexing	`tuple.[0]`	`tuple.[0]`
Tuple Unpacking	`tuple.[1..]...`	`tuple.[1..]~...`
Full Tuple Unpacking	`tuple.[..]...`	`tuple~...`

And then two good choices for disambiguation for dependent nested packs:

— `obj. ...elems`, or

— `obj.pack elems`.

And lastly I think the question of packs outside of templates could be best solved by requiring a preceding pack declaration in scope - where `using ...;` could be a no-op pack declaration that counts for that rule.

This proposal deals exclusively with syntax. The semantics of what some of these options mean (in particular, how does tuple indexing evaluate) was discussed in [P1858R2].

6 References

[N4235] Daveed Vandevoorde. 2014-10-10. Selecting from Parameter Packs.
https://wg21.link/n4235

[P0780R2] Barry Revzin. 2018-03-14. Allow pack expansion in lambda init-capture.
https://wg21.link/p0780r2

[P1061R2] Barry Revzin, Jonathan Wakely. 2022-04-22. Structured Bindings can introduce a Pack.
https://wg21.link/p1061r2

[P1240R2] Daveed Vandevoorde, Wyatt Childers, Andrew Sutton, Faisal Vali. 2022-01-14. Scalable Reflection.
https://wg21.link/p1240r2

[P1306R1] Andrew Sutton, Sam Goodrick, Daveed Vandevoorde. 2019-01-21. Expansion statements.
https://wg21.link/p1306r1

[P1858R2] Barry Revzin. 2020-03-01. Generalized pack declaration and usage.
https://wg21.link/p1858r2

[P2277R0] Barry Revzin. 2021-01-03. Packs outside of Templates.
https://wg21.link/p2277r0

---

1. There, the facility was trying to support iterating over both packs and tuples, but it can be ambiguous in some contexts. If the direction were to iterate over `tuple` as a tuple and `{pack...}` as a pack, we could get both pieces of functionality with any ambiguity and without having to construct a tuple out of the pack.↩

2. But they *do* use more braces, so we can advertise it as uniform pack expansion.↩

3. Spoiler alert, having a dedicated language feature doesn't just look much better, it also compiles tremendously faster.↩

4. This implementation requires non-transient constexpr allocation, which currently doesn't work, but can be rewritten to avoid it. And besides, the whole reflection part doesn't currently work either.↩

5. D's slice overloading is fairly involved, and also supports adding multiple groups. Like `x[1, 2, 8..20]`. On the one hand, this is interesting, but on the other hand with the adoption of multi-dimensional subscript operators, we're establishing a meaning for `x[1, 2]` in C++ that is at odds with interpreting this as a slice of the 2nd and 3rd elements.↩

6. Six dots to pull one element of a tuple? That's more than a little excessive.↩

7. Yeah, `std::slice` exists.↩